

ALJANNAT Luxury Fashion E-commerce Website

Document type: Software Architecture & Project Documentation

Project: ALJANNAT Luxury Fashion E-commerce Website

Author: Principal Software Architect

Version: 1.0

Date: 2026-08-02

Table of Contents

1. Project Overview
 2. Complete Website Blueprint
 3. Complete Website Structure
 4. Frontend Documentation
 5. Backend Documentation
 6. Database Documentation
 7. API Documentation
 8. Authentication Flow
 9. Product Management Guide
 10. Image Management Guide
 11. Payment & Invoice Guide
 12. Website Data Flow
 13. File Dependency Map
 14. Project Status
 15. Deployment Guide
 16. Maintenance Guide
 17. Visual Diagrams
 18. Appendices
-

1. Project Overview

1.1 Purpose of the website

ALJANNAT is a luxury fashion storefront designed to showcase premium women's dresses and boutique apparel. It aims to provide an elegant, polished shopping experience with an aspirational brand tone and a responsive, modern interface.

1.2 Business goals

- Demonstrate a full e-commerce capability for a luxury fashion brand
- Convert browsing into cart and checkout actions
- Support guest shopping with backend persistence
- Provide wishlist, cart, order, payment, and invoice workflows
- Establish a maintainable architecture for future expansion

1.3 Target audience

- Luxury fashion customers seeking bridal, evening, formal, and party dresses
- Mobile-first shoppers with modern browsers
- Fashion buyers who value aesthetic storytelling and premium experience
- Developers and operations teams maintaining a full-stack fashion storefront

1.4 Main features

- Homepage hero slider and curated collections
- Product listing and collection filtering
- Product detail pages with images and description
- Wishlist and cart support
- Backend persistence for cart and wishlist
- Order creation and payment flow
- Invoice creation and PDF generation backend service
- Authentication endpoints for registered users
- Admin endpoints for order and payment management

1.5 Technologies used

- Frontend: HTML5, CSS3, JavaScript ES6
- Styling: responsive CSS, animations, hero sections
- Backend: Node.js, Express
- Database: MongoDB via Mongoose
- Security: Helmet, CORS, rate limiting, cookie parser
- PDF generation: pdfkit, qrcode
- File upload: multer
- Validation: joi

1.6 Overall system architecture

This repository is organized as a hybrid static frontend plus an Express backend. The backend serves the static pages, exposes REST APIs under `/api`, and persists business data in MongoDB.

- Frontend: `index.html`, `collections.html`, `product.html`, `cart.html`, `wishlist.html`, `contact.html`, `about.html`, `search.html`
- Backend: `backend/src/index.js` and `routes/controllers`
- Database models: `backend/src/models/*.js`

1.7 Current project status

The website contains:

- Fully implemented static frontend pages for the storefront
- Backend API endpoints for cart, wishlist, orders, payments, authentication
- MongoDB connection and persistence
- Invoice PDF generation service
- Product catalogue implemented in frontend JS data

1.8 Completed features

- Static homepage and collections pages
- Product pages with detail view
- Cart and wishlist UI
- Guest session support through `X-Session-Id`
- Backend cart and wishlist persistence
- Orders and payments backend endpoints
- Invoice generation service
- Secure backend middleware

1.9 Remaining features

- Product API endpoints (`GET /api/products`, category APIs)
- Full admin panel UI
- Authentication pages (login/register/account/profile)
- Checkout page UI and order confirmation page
- Search results page integration with backend APIs
- Category and collection backend management APIs
- Coupons, reviews, address book, notifications, audit logs

1.10 Future improvements

- Convert frontend product catalog from `static js/products.js` to backend-driven API
- Add user account pages and persistent login
- Build an admin dashboard for product and order management
- Enable email verification and password reset
- Add analytics, monitoring, and structured logging
- Harden MongoDB with authentication and TLS for production

2. Complete Website Blueprint

2.1 Overall project architecture

The ALJANNAT solution is a hybrid static-and-API website.

- Static frontend assets live at repository root
- JavaScript controls UI state and requests backend APIs

- Express backend exposes REST API routes and serves static files
- MongoDB stores application data such as cart, wishlist, orders, payments, users, invoices

2.2 Frontend architecture

The frontend is built as a static multi-page application.

- Global layout and navigation are shared across pages
- Page-specific data-page attributes instruct the JS runtime
- `js/app.js` initializes page behavior, hero slider, search overlay, wishlist/cart counts, and page-specific renderers
- `js/products.js` defines product catalog data and rendering helpers
- `js/storage.js` abstracts `localStorage` and API fallback
- `js/cart.js` implements cart UI and add/remove actions
- `js/wishlist.js` manages saved products and wishlist count
- `js/search.js` provides on-page search interactions
- `js/filters.js` handles collections filtering and sorting
- `js/payment.js` contains the checkout modal and payment submission workflow

2.3 Backend architecture

The backend uses an MVC-inspired pattern.

- `backend/src/index.js`: Express app, middleware, routes, static file serving
- `backend/src/config/db.js`: MongoDB connection helper
- `backend/src/routes/*.route.js`: route registration for API endpoints
- `backend/src/controllers/*.controller.js`: business logic for APIs
- `backend/src/models/*.js`: Mongoose schemas and collections
- `backend/src/middleware/*.js`: authentication, admin guard, upload handling, error handling
- `backend/src/services/*.js`: invoice and PDF generator services
- `backend/src/utils/*.js`: identity and sequence utilities

2.4 Database architecture

MongoDB stores collections modeled by Mongoose.

- `users` and `admins` manage authentication and roles
- `products` stores catalog detail
- `cart` and `wishlist` persist session or user selections
- `orders`, `payments`, and `invoices` capture purchase lifecycle
- `contact` and `newsletter` support visitor interactions
- `bankaccount`, `easypaisa` support payment instructions
- `counter` supports sequence generation

2.5 API architecture

The REST API is grouped by business domain.

- `/api/auth` for authentication
- `/api/cart` for cart CRUD
- `/api/wishlist` for wishlist toggles
- `/api/orders` for order creation, retrieval, invoice download, and admin operations
- `/api/payments` for payment submission, verification, and proof upload

2.6 Authentication flow

Current backend supports JWT-based authentication with token verification middleware. Registered users can authenticate; anonymous visitors use session IDs for cart/wishlist persistence.

2.7 Shopping flow

1. Visitor browses homepage or collections
2. Visitor adds products to cart or wishlist
3. Cart is persisted via API or fallback `localStorage`
4. Visitor opens cart and proceeds to checkout
5. Order placement triggers backend order creation and invoice generation

2.8 Checkout flow

- Order creation is performed with `POST /api/orders`
- The backend validates order data with Joi
- A unique order number and invoice number are generated

- The cart is cleared after order creation

2.9 Payment flow

- Payment requests submit to `POST /api/payments/process`
- The backend creates a payment record and associates it to the order
- Admin verification can occur through `POST /api/payments/verify`
- Payment status is updated on both payment and order records

2.10 Invoice flow

- Invoices are modeled in `backend/src/models/invoice.model.js`
- `backend/src/services/invoice.service.js` generates PDF invoices using `pdfkit`
- Invoice PDFs are stored under `uploads/invoices` or the configured path
- Retrieve invoice PDFs with `GET /api/orders/:id/invoice`

2.11 Admin panel flow

The backend includes admin-only route guards for order and shipping status management, but the frontend admin panel UI is not implemented in this repository.

3. Complete Website Structure

3.1 Implemented pages

Page	Path	Purpose	Status
Home	<code>index.html</code>	Brand introduction, hero slider, featured collections, product previews	Implemented
Collections	<code>collections.html</code>	Product browsing, filtering, sorting	Implemented
Product Details	<code>product.html</code>	Product information, gallery, add to cart, wishlist button	Implemented
Wishlist	<code>wishlist.html</code>	Saved favorites page	Implemented
Cart	<code>cart.html</code>	Shopping cart review and order summary	Implemented
Contact	<code>contact.html</code>	Contact and boutique information	Implemented
Search	<code>search.html</code>	Search interface with results	Implemented
About	<code>about.html</code>	Brand story and company information	Implemented

3.2 Planned or not present pages

Page	Path	Purpose	Status
Checkout	<code>checkout.html</code>	Final payment form and order confirm	Not implemented
Order Confirmation	<code>order-confirmation.html</code>	Display order details after checkout	Not implemented
Invoice	<code>invoice.html</code>	Downloadable invoice viewer	Not implemented
Account	<code>account.html</code>	Customer account dashboard	Not implemented
Login	<code>login.html</code>	User login	Not implemented
Register	<code>register.html</code>	User registration	Not implemented
Forgot Password	<code>forgot-password.html</code>	Password reset request	Not implemented
Profile	<code>profile.html</code>	Manage user profile and addresses	Not implemented
Orders	<code>orders.html</code>	Historic orders list	Not implemented
Admin Dashboard	<code>admin/dashboard.html</code>	Admin operations and reporting	Not implemented
Settings	<code>admin/settings.html</code>	Admin configuration	Not implemented

3.3 Page-specific documentation

Home

- Purpose: Brand presentation and product discovery
- Components: Hero slider, category strip, featured product grids, promo banner, newsletter card
- User actions: Browse collections, add items to cart, search, open wishlist/cart
- Connected APIs: `GET /api/cart`, `GET /api/wishlist` via frontend count updates
- Database collections: `Cart`, `Wishlist`, optionally `Product` once back-end product catalog is added

Shop / Collections

- Purpose: Allow shoppers to browse products by category, collection, price, size
- Components: filters panel, sort menu, grid view, product cards, pagination controls
- User actions: Filter selection, sorting, toggle view, quick add, quick view
- Connected APIs: currently none; planned `GET /api/products` and query endpoints

- Database collections: `Product` (planned), `Category`, `Collection` (planned)

Product Details

- Purpose: Show product imagery, description, pricing, available sizes and colors
- Components: gallery, detail blocks, add-to-cart button, wishlist button, related products
- User actions: add to cart, add to wishlist, view different images, read care instructions
- Connected APIs: cart and wishlist API via `addToCart()` and `toggleWishlist()`; future product API
- Database collections: `Product`, `Cart`, `Wishlist`

Wishlist

- Purpose: Persist favorite products for later purchase
- Components: wishlist grid with product cards and remove actions
- User actions: toggle wishlist item, view saved products, navigate to product details
- Connected APIs: `GET /api/wishlist`, `PUT /api/wishlist/:productId`
- Database collections: `Wishlist`, `Product`

Cart

- Purpose: Review selected items and initiate checkout
- Components: cart item list, summary panel, subtotal/total display, checkout button
- User actions: remove cart item, view order totals, proceed to payment flow
- Connected APIs: `GET /api/cart`, `PUT /api/cart/:productId`, `DELETE /api/cart/:productId`
- Database collections: `Cart`, `Order`

Contact

- Purpose: Provide contact details and boutique location
- Components: address card, phone/email list
- User actions: view support details and contact the brand
- Connected APIs: none in current implementation
- Database collections: none (planned `Contact` if message submission is added)

Search

- Purpose: Search product catalog from any page
- Components: overlay, input field, result list
- User actions: type keywords, navigate to product detail pages
- Connected APIs: none; search is performed on static client-side product data currently
- Database collections: planned `Product`

Missing pages (Account, Login, Register, etc.)

These pages are planned for future authenticated user workflows. They would connect to `Auth`, `Order`, and `User` systems.

4. Frontend Documentation

4.1 Frontend folder structure

```
/aljannat
/css
  animations.css
  responsive.css
  style.css
/js
  app.js
  cart.js
  filters.js
  payment.js
  products.js
  search.js
  storage.js
  wishlist.js
/assets
  fonts/
  icons/
  images/
  videos/
index.html
collections.html
product.html
cart.html
wishlist.html
contact.html
search.html
about.html
```

4.2 Important frontend files

index.html

- Brand landing page
- Hero slider and featured product sections
- Navigation and search overlay

collections.html

- Product catalog browsing and filtering
- Sort menu and toggle view
- Filter controls for category, collection, price, size

product.html

- Product detail layout and gallery
- Add to cart and wishlist actions
- Related product recommendations

cart.html

- Cart review and order summary
- Checkout button
- Cart total calculations

wishlist.html

- Favorite products list
- Product card reuse

contact.html

- Contact and boutique information

js/app.js

- Global page initialization
- Hero slider setup
- Theme toggle and sticky header
- Modal and search overlay behavior

- Page-specific render functions

`js/storage.js`

- API request wrapper
- LocalStorage fallback for cart/wishlist/session
- Session management with `aljannat_session_id`
- Headers and fetch helper

`js/products.js`

- Product catalog data model
- Card renderer and product grid renderer
- Price formatting helper

`js/cart.js`

- Cart item lifecycle
- `addToCart()` and `removeCartItemHandler()`
- `renderCart()` and `updateCartCount()`
- Cart UI event delegation

`js/wishlist.js`

- Wishlist fetch and toggle logic
- `renderWishlist()` and `updateWishlistCount()`
- Wishlist event delegation

`js/search.js`

- Client-side query search
- Search overlay open/close logic
- Result rendering

`js/filters.js`

- Collection filtering logic
- URL parameter driven filters
- Sorting by price/name
- List view toggle

`js/payment.js`

- Checkout modal and payment workflow
- Payment method selection
- Payment API submission and verification
- Payment result confirmation screen
- Note: this is UI logic; actual backend payment API is partially implemented

4.3 HTML structure

- Each page uses a shared header and footer structure
- A `data-page` attribute is applied to `<body>` to identify the current page
- Shared layout classes such as `.site-shell`, `.container`, and `.page-section` are reused across pages
- Search overlay and toast notifications are appended to every page

4.4 CSS organization

- `style.css` contains the core brand styles, grids, layout, and component styling
- `responsive.css` contains breakpoints for mobile/tablet support
- `animations.css` contains reveal animations, hover transitions, and loader animation
- Shared classes include `.product-card`, `.btn`, `.site-header`, `.mega-menu`, `.hero`, and `.site-footer`

4.5 JavaScript modules and relationships

- `app.js` is the root initializer for all frontend pages
- `storage.js` is the data access abstraction used by cart and wishlist modules
- Product rendering is centralized in `products.js`
- Cart and wishlist modules consume the shared product catalog and storage layer
- Search and filters interact with the same `products` array for client-side results

- `payment.js` is separate but communicates via the cart total and backend payment endpoints

4.6 Component relationships

- navbar and footer are present in each HTML page and are not currently componentized in JS
- product cards are created via `createProductCard()` in `products.js`
- hero section is managed via `setupHeroSlider()` in `app.js`
- search overlay is managed by `bindSearchUI()` in `search.js`
- filters are managed by `filters.js`, with filter controls and result rendering
- wishlist and cart UI counts are updated by `updateWishlistCount()` and `updateCartCount()` across pages

4.7 Event handling

- Global document click handler in `cart.js` for add/remove actions
- Global document click handler in `wishlist.js` for wishlist toggles
- `search.js` binds input events for live search
- `filters.js` binds filter control change events and sort events
- `app.js` binds navigation hovering, back-to-top click, theme toggle, and modal events

4.8 API integration

- `storage.js` uses `apiFetch()` to call `/api/*` endpoints with session headers
- Cart operations call `GET /api/cart`, `PUT /api/cart/:productId`, and `DELETE /api/cart/:productId`
- Wishlist operations call `GET /api/wishlist` and `PUT /api/wishlist/:productId`
- Payment operations call `POST /api/payments/process` and `POST /api/payments/verify`

4.9 Responsive design

- The CSS uses breakpoints for mobile, tablet, and desktop
- Navigation collapses into a hamburger menu on narrow screens
- Product grids and hero layouts adapt with flexible columns
- Cart and product detail layouts stack vertically on smaller screens

4.10 Animations

- Page reveal animations use the `.reveal` class and `IntersectionObserver` in `app.js`
- Hero slider transitions with active slide toggling
- Buttons and cards include hover animations for a premium feel
- Loader state appears while the page initializes

4.11 Frontend responsibilities summary

- Navbar: shared HTML and JS in `app.js`
- Footer: shared HTML only
- Hero section: `app.js`
- Product cards: `products.js`
- Product details: `product.html`, `app.js`
- Search: `search.js`
- Filters: `filters.js`, `collections.html`
- Wishlist: `wishlist.js`
- Cart: `cart.js`, `cart.html`
- Checkout: `payment.js` (UI modal)
- Invoice: backend service only; no frontend invoice page implemented
- User authentication: backend API only; no frontend login/register pages

5. Backend Documentation

5.1 Backend folder structure


```
/backend
.env.example
Dockerfile
package.json
src/
  config/
    db.js
  controllers/
    auth.controller.js
    cart.controller.js
    order.controller.js
    payment.controller.js
    wishlist.controller.js
  middleware/
    admin.middleware.js
    auth.middleware.js
    error.middleware.js
    upload.middleware.js
  models/
    bankaccount.model.js
    cart.model.js
    contact.model.js
    counter.model.js
    easypaisa.model.js
    invoice.model.js
    order.model.js
    payment.model.js
    product.model.js
    user.model.js
    wishlist.model.js
    newsletter.model.js
  routes/
    auth.route.js
    cart.route.js
    order.route.js
    payment.route.js
    wishlist.route.js
  services/
    invoice.service.js
    pdf.service.js
  utils/
    identity.util.js
    sequence.util.js
  index.js
  uploads/
```

5.2 File descriptions

backend/src/index.js

- Main Express server entry
- Loads environment variables
- Connects to MongoDB
- Applies security and body parsing middleware
- Registers API routes
- Serves frontend static assets from the repository root
- Handles fallback route and error middleware

backend/src/config/db.js

- MongoDB connection helper using Mongoose
- Uses `useNewUrlParser` and `useUnifiedTopology`
- Exits process on connection failure

backend/src/routes/*.route.js

- Define route paths and methods
- Map requests to controller functions
- Keep route definitions small and expressive

backend/src/controllers/*.controller.js

- Implement business logic and request validation
- `auth.controller.js`: user registration, login, profile retrieval
- `cart.controller.js`: cart retrieval, add/update item, item removal
- `wishlist.controller.js`: wishlist retrieval and toggle functionality
- `order.controller.js`: order creation, retrieval, invoice generation, admin order management
- `payment.controller.js`: payment processing, verification, proof upload

backend/src/middleware/auth.middleware.js

- JWT authorization middleware
- Accepts token from `Authorization` header or cookies
- Attaches `req.user` to authenticated requests

backend/src/middleware/admin.middleware.js

- Verifies `req.user.role === 'admin'`
- Protects admin routes

backend/src/middleware/error.middleware.js

- Handles 404 responses
- Formats error responses with status code and details

backend/src/middleware/upload.middleware.js

- Configures `multer` for file upload storage
- Accepts JPEG, PNG, WEBP, and PDF files
- Writes uploads to `uploads/`
- Enforces 5 MB file size limit

backend/src/models/*.js

- Mongoose collection schemas
- Enforce field validation and indexes
- Provide relationships via `ref`

backend/src/services/invoice.service.js

- Creates invoice document and PDF
- Uses company info from environment variables
- Delegates PDF generation to `pdf.service.js`

backend/src/services/pdf.service.js

- Uses `pdfkit` to generate invoice PDFs
- Creates invoice layout, line items, and QR code
- Writes PDF file to `uploads/invoices`

backend/src/utls/identity.util.js

- Normalizes request identity from authenticated user or session ID
- Used by cart, wishlist, orders, and payments

backend/src/utls/sequence.util.js

- Generates sequential order and invoice numbers
- Uses `Counter` collection for atomic increment

5.3 Request lifecycle

1. Browser request arrives at Express server
2. `index.js` applies middleware and logs request
3. Route path matches one of `/api/*`
4. Controller function executes business logic
5. Validation and identity checks occur
6. Controller reads or writes MongoDB through models
7. Response is returned as JSON

- 8. Error middleware catches exceptions and formats the error

5.4 Backend request flow example

- POST /api/orders
 - order.route.js routes request to createOrder
 - order.controller.js validates payload with Joi
 - sequence.util.js generates order/invoice numbers
 - Order.create() writes order document
 - Cart.findOneAndDelete() clears cart
 - invoice.service.js generates invoice document and PDF
 - Response returns order and invoice metadata
-

6. Database Documentation

6.1 Actual MongoDB collections

Users

- Purpose: Registered users and admin accounts
- Fields:
 - _id, name, email, password, role, createdAt
- Relationships: orders, payments
- Indexes: email unique
- Example document:

```
{
  "_id": "64...",
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "<hashed>",
  "role": "customer",
  "createdAt": "2026-08-02T..."
}
```

Products

- Purpose: Catalog of dresses and apparel
- Fields: name, category, collection, price, salePrice, rating, colors, sizes, stock, image, hoverImage, badge, description, createdAt, updatedAt
- Relationships: cart items, wishlist items, order line items
- Indexes: none currently defined
- Example document:

```
{
  "name": "Aurora Silk Dress",
  "category": "party",
  "collection": "luxury",
  "price": 620,
  "salePrice": 540,
  "rating": 4.9,
  "colors": ["Gold", "Black"],
  "sizes": ["XS", "S", "M", "L"],
  "stock": "In Stock",
  "image": "https://...",
  "hoverImage": "https://...",
  "badge": "Limited Edition",
  "description": "A sculpted silk dress..."
}
```

Cart

- Purpose: Persist shopping cart contents for guest or authenticated sessions
- Fields: user, sessionId, items, updatedAt
- Relationships: User, Product
- Indexes: user, sessionId

- Example document:

```
{
  "user": null,
  "sessionId": "session-abc123",
  "items": [{ "product": "1", "quantity": 1 }],
  "updatedAt": "2026-08-02T..."
}
```

- Note: product is stored as Mixed to accept frontend string IDs and ObjectIds

Wishlist

- Purpose: Persist customer favorites
- Fields: user, sessionId, products, updatedAt
- Relationships: User, Product
- Indexes: user, sessionId
- Example document:

```
{
  "sessionId": "session-abc123",
  "products": ["64...productId"],
  "updatedAt": "2026-08-02T..."
}
```

Orders

- Purpose: Store purchase orders and lifecycle state
- Fields: user, sessionId, orderNumber, invoiceNumber, items, subtotal, shippingCost, tax, discount, total, currency, customer, payment, shippingStatus, orderStatus, createdAt
- Relationships: User, Product, Payment, Invoice
- Indexes: orderNumber, invoiceNumber, user, sessionId
- Example document:

```
{
  "orderNumber": "ORD-2026-000001",
  "invoiceNumber": "INV-2026-000001",
  "items": [{ "productId": "64...", "name": "Aurora Silk Dress", "quantity": 1, "unitPrice": 540,
"totalPrice": 540 }],
  "subtotal": 540,
  "shippingCost": 0,
  "tax": 0,
  "discount": 0,
  "total": 540,
  "customer": {
    "name": "Jane Doe",
    "email": "jane@example.com",
    "shippingAddress": "..."
  },
  "payment": {
    "method": "cod",
    "status": "pending"
  },
  "shippingStatus": "pending",
  "orderStatus": "pending"
}
```

Payments

- Purpose: Store payment records and verification state
- Fields: order, user, sessionId, method, status, amount, transactionId, transactionDate, paymentDetails, proof, verification, createdAt
- Relationships: Order, User
- Indexes: user, sessionId, order
- Example document:

```
{
  "order": "64...orderId",
  "method": "card",
  "status": "pending_verification",
  "amount": 540,
  "transactionId": "txn-123",
  "paymentDetails": { "accountName": "...", },
  "proof": [],
  "verification": { "notes": "" }
}
```

Invoice

- **Purpose:** Persist invoice metadata and PDF path
- **Fields:** order, invoiceNumber, pdfPath, status, currency, totals, billingAddress, shippingAddress, customer, company, createdAt
- **Relationships:** Order
- **Example document:**

```
{
  "order": "64...orderId",
  "invoiceNumber": "INV-2026-000001",
  "pdfPath": "uploads/invoices/INV-2026-000001.pdf",
  "status": "issued",
  "totals": { "subtotal": 540, "shipping": 0, "tax": 0, "discount": 0, "total": 540 },
  "billingAddress": "...",
  "customer": { "name": "Jane Doe", "email": "jane@example.com" }
}
```

Contact

- **Purpose:** Store inbound contact form messages if implemented
- **Fields:** name, email, subject, message, createdAt
- **Relationships:** none
- **Example:** not currently used by frontend

Newsletter

- **Purpose:** Store subscriber email addresses
- **Fields:** email, subscribedAt
- **Example document:**

```
{
  "email": "jane@example.com",
  "subscribedAt": "2026-08-02T..."
}
```

BankAccount and EasyPaisa

- **Purpose:** Payment instruction storage
- **Fields:** bank/easy paisa account details and QR code metadata
- **Relationship:** Payment or frontend display only

Counter

- **Purpose:** Sequence generation for orders and invoices
- **Fields:** name, sequence
- **Example document:**

```
{
  "name": "order",
  "sequence": 10
}
```

6.2 Planned collections

The following collections are not implemented but are recommended for a complete e-commerce platform:

- categories
- collections

- orderItems (separate from orders for normalization)
- reviews
- coupons
- addresses
- notifications
- auditLogs
- siteSettings

6.3 Relationships and ER-style overview

```
[Users] 1 --- * [Orders]
[Users] 1 --- * [Payments]
[Orders] 1 --- 1 [Invoice]
[Orders] * --- * [Products] (via embedded order items)
[Cart] * --- * [Products]
[Wishlist] * --- * [Products]
```

7. API Documentation

7.1 Authentication

POST /api/auth/register

- Request body: { name, email, password }
- Returns: { success, data: { id, name, email, role }, token }
- Validation: Joi requires email, password min 8 characters
- Notes: Registers user and returns JWT

POST /api/auth/login

- Request body: { email, password }
- Returns: { success, data: { id, name, email, role }, token }
- Notes: Authenticates user and issues JWT

GET /api/auth/me

- Authentication: Bearer token or cookie required
- Returns: authenticated user profile

7.2 Cart

GET /api/cart

- Purpose: retrieve current cart contents for session or authenticated user
- Headers: X-Session-Id or auth cookie
- Response: { success, data: [{ product, quantity }] }

PUT /api/cart/:productId

- Purpose: add or update cart item
- Request body: { quantity }
- Returns: updated cart items
- Validation: requires productId and quantity >= 1

DELETE /api/cart/:productId

- Purpose: remove a product from the cart
- Returns: updated cart items

7.3 Wishlist

GET /api/wishlist

- Purpose: retrieve wishlist for current session or authenticated user
- Response: product ID array

PUT /api/wishlist/:productId

- Purpose: toggle wishlist item presence
- Returns: updated product ID array

7.4 Orders

POST /api/orders

- Purpose: create an order
- Request body:
 - items: array with productId, name, quantity, unitPrice, totalPrice
 - subtotal, shippingCost, tax, discount, total, currency
 - customer: name, email, phone, shippingAddress, billingAddress
 - paymentMethod
- Returns: created order and invoice

GET /api/orders

- Purpose: retrieve orders for session or authenticated user

GET /api/orders/:id

- Purpose: retrieve a single order by ID
- Includes authorization checks for user or session

GET /api/orders/:id/invoice

- Purpose: download invoice PDF for an order
- Returns: PDF file stream

GET /api/orders/admin/all

- Purpose: admin order list
- Authentication: authGuard and adminGuard

PUT /api/orders/admin/:id/status

- Purpose: update order status
- Request body: { status }

PUT /api/orders/admin/:id/shipping

- Purpose: update shipping status
- Request body: { status }

7.5 Payments

POST /api/payments/process

- Purpose: create payment record for an order
- Request body: orderId, amount, paymentMethod, transactionId, transactionDate, paymentDetails
- Returns: payment resource

POST /api/payments/verify

- Purpose: admin verification of payment
- Request body: paymentId, status, notes

POST /api/payments/proof/:paymentId

- Purpose: upload proof files for a payment
- Authentication: required
- Uploads: multipart form with files

7.6 Missing or planned APIs

The following APIs are planned but not implemented in this repository:

- GET /api/products

- GET /api/products/:id
- GET /api/categories
- GET /api/collections
- POST /api/contact
- POST /api/newsletter
- POST /api/auth/forgot-password
- POST /api/auth/reset-password
- GET /api/admin/products
- PUT /api/admin/products/:id
- DELETE /api/admin/products/:id

8. Authentication Flow

8.1 Guest user flow

- Frontend generates a session ID using `js/storage.js` and stores it in `localStorage` under `aljannat_session_id`
- The session ID is included in every API request via `X-Session-Id`
- Backend resolves cart, wishlist, and orders by `sessionId` if no authenticated user is present

8.2 Registered user flow

- User registers via `POST /api/auth/register`
- Backend hashes password with `bcrypt` and stores the user
- Backend returns a JWT token
- User logs in via `POST /api/auth/login`
- Authenticated requests include JWT in header `Authorization: Bearer <token>` or cookie
- `auth.middleware.js` verifies JWT and loads `req.user`

8.3 JWT

- Tokens are signed with `JWT_SECRET`
- Token expiry is configured by `JWT_EXPIRES_IN`
- Backend uses the payload `id` field to identify user

8.4 Refresh tokens

- Not currently implemented in the existing backend
- Recommended future enhancement: add `refreshToken` storage and route

8.5 Secure cookies

- Backend supports cookie usage via `cookie-parser`
- No secure cookie session implementation is currently present
- Recommend using `SESSION_COOKIE_SECURE=true` and `httpOnly` cookies in production

8.6 Email verification

- Not implemented
- Recommended: email token generation, verification link, `isVerified` field on User

8.7 Password reset

- Not implemented
- Recommended: `forgot-password` and `reset-password` endpoints with secure tokens

8.8 Role-based access

- `auth.middleware.js` checks authentication
- `admin.middleware.js` checks `req.user.role === 'admin'`
- Used by admin order endpoints

8.9 Session flow diagram

```
[Visitor] -> [Browser] -> [Frontend JS]
-> sessionId generated -> localStorage
-> API call with X-Session-Id
-> [Backend] -> resolve Cart/Wishlist by sessionId
-> MongoDB
-> Response
```

9. Product Management Guide

9.1 Add a new product

1. Add a document to `products` collection using `backend/src/models/product.model.js`
2. Fields required: `name`, `category`, `collection`, `price`, `stock`, `image`, `hoverImage`
3. Optional: `salePrice`, `rating`, `colors`, `sizes`, `badge`, `description`
4. Once backend product API is created, use `POST /api/admin/products`

9.2 Edit a product

- Modify the product document in MongoDB or via the future admin API
- Update `product.model.js` if adding new product attributes
- Ensure frontend page `product.html` and `products.js` rendering support new attributes

9.3 Delete a product

- Remove the document from `products` collection
- Clear references in `carts`, `wishlists`, and `orders` if needed
- Future admin endpoint should support `DELETE /api/admin/products/:id`

9.4 Upload images

- Stored via `upload.middleware.js` into the configured `uploads/` folder
- If product images are stored locally, save path in `product.image` and `product.hoverImage`
- Currently frontend uses external URLs, so image upload is not fully wired into the UI

9.5 Change prices

- Update price fields in product documents
- If `salePrice` exists, frontend displays it as current price and `price` as old price

9.6 Add colors and sizes

- Add arrays to `colors` and `sizes` in product documents
- Frontend renders these arrays into pills and quick-add actions

9.7 Manage inventory

- `stock` is currently a string field such as `In Stock`, `Low Stock`, `Pre-Order`
- Recommended future improvement: add numerical inventory count and availability logic

9.8 Create categories and collections

- Frontend filters depend on `category` and `collection` values in product objects
- To support backend-managed categories, implement a `Category` collection and routing

9.9 Mark Featured and Bestseller

- Use `badge` field on product documents
- Possible values: `New`, `Luxury`, `Bridal`, `Sale`, `Limited Edition`

9.10 Add SEO information

- Add structured metadata to products such as meta title and description
- Use page-level `<meta>` tags in `product.html` and `collections.html`
- Future backend should store SEO fields in `Product` documents

10. Image Management Guide

10.1 Storage location

- Uploaded files are stored under `uploads/`
- Invoices and payment proofs are stored under `uploads/invoices` and `uploads/`

10.2 Upload process

- Upload middleware `upload.middleware.js` handles destination and filename

- Files are saved with timestamped, sanitized names
- Allowed MIME types: image/jpeg, image/png, image/webp, application/pdf

10.3 URL generation

- Backend should expose uploaded files via static serving or a dedicated GET /uploads/... route
- Currently, frontend product images are external asset URLs, not uploads

10.4 Recommended image sizes

- Hero/collection banners: 1600x900 or wider
- Product thumbnails: 900x900
- Gallery images: 900x900 to 1200x1200
- Optimize for web delivery with webp or compressed jpeg

10.5 Folder structure

```
/uploads
  invoices/
  payment-proofs/
  products/ (planned)
```

10.6 Optimization recommendations

- Serve responsive image sizes with srcset
- Create thumbnails for catalog lists
- Use lazy loading on all product media
- Consider a CDN for production image delivery

10.7 Naming conventions

- Use lowercase, hyphen-separated filenames
- Include timestamp and sanitized product identifier
- Example: 20260802-aurora-silk-dress-1.jpg

11. Payment & Invoice Guide

11.1 Checkout flow

- User opens checkout modal via cart summary
- Payment method selected from available options
- Card details collected if card payment is chosen
- Backend POST /api/payments/process is called
- Payment verification can be triggered with POST /api/payments/verify

11.2 Payment methods

Supported payment methods in the codebase:

- cod (Cash on Delivery)
- bank_transfer
- easypaisa
- card
- paypal

Actual UI currently implements card and COD selection; the backend supports all five methods in schema.

11.3 Payment verification

- payment.controller.js tracks verification state
- POST /api/payments/verify updates payment status and order status
- Payment status values: pending, pending_verification, verified, paid, failed, rejected, refunded

11.4 Invoice generation

- invoice.service.js creates invoice records and generates PDF files
- pdf.service.js uses pdfkit and optional QR code generation
- Invoice PDFs are saved and returned via download route

11.5 Order creation

- Orders are created in `Order` model with embedded order item details
- Order creation also generates invoice numbers and can create payment records for COD
- Cart content is cleared after order creation

11.6 Order status

- Order status values: pending, confirmed, processing, packed, shipped, delivered, cancelled, returned
- Shipping status values: pending, ready, dispatched, in_transit, delivered

11.7 Payment status

- Payments maintain status changes separately from orders
- The order payment object mirrors status for convenience

11.8 Shipping status

- Shipping status is updated via admin endpoints
- The frontend currently does not expose shipping updates

11.9 PDF invoice generation

- Invoice number sequence is generated with `Counter`
- PDFs are created with company and customer details
- The generated PDF path is stored in the invoice document

11.10 Admin verification

- Admins can verify payments and update order status through backend routes
- Future admin UI should expose these operations in a dashboard

12. Website Data Flow

12.1 Visitor flow

```
[Visitor] -> [Browser] -> [Frontend]
-> Loads HTML/CSS/JS
-> Reads `js/products.js` catalog
-> Generates or reuses sessionId
-> Updates UI counts
```

12.2 Frontend to API flow

```
[Frontend] -> API request
-> `storage.js` adds X-Session-Id
-> Calls `/api/cart`, `/api/wishlist`, `/api/orders`, `/api/payments`

[Backend] -> Controller -> Model -> MongoDB -> Response
```

12.3 Backend to database flow

- Controllers validate incoming requests
- Models map documents to collections
- Queries use `find`, `create`, `findOneAndUpdate`, `findOneAndDelete`
- Responses return JSON to frontend

12.4 Response to frontend UI

- Frontend receives JSON success responses
- UI modules update cart/wishlist counts and render new state
- Checkout modal displays processing state and confirmation

13. File Dependency Map

13.1 Product page dependency

```
product.html
-> js/products.js
-> js/app.js
-> js/cart.js
-> js/wishlist.js
-> js/search.js
-> js/storage.js
-> js/payment.js
```

13.2 Cart feature dependency

```
cart.html
-> js/cart.js
-> js/storage.js
-> js/products.js
-> backend/src/routes/cart.route.js
-> backend/src/controllers/cart.controller.js
-> backend/src/models/cart.model.js
```

13.3 Wishlist feature dependency

```
wishlist.html
-> js/wishlist.js
-> js/storage.js
-> js/products.js
-> backend/src/routes/wishlist.route.js
-> backend/src/controllers/wishlist.controller.js
-> backend/src/models/wishlist.model.js
```

13.4 Order and payment dependency

```
payment.js
-> js/storage.js
-> backend/src/routes/payment.route.js
-> backend/src/controllers/payment.controller.js
-> backend/src/models/payment.model.js
-> backend/src/controllers/order.controller.js
-> backend/src/services/invoice.service.js
-> backend/src/services/pdf.service.js
```

13.5 Authentication dependency

```
frontend login/register pages (planned)
-> backend/src/routes/auth.route.js
-> backend/src/controllers/auth.controller.js
-> backend/src/models/user.model.js
-> backend/src/middleware/auth.middleware.js
```

13.6 System dependency chain

```
Page -> Frontend JS -> storage.js -> API endpoint -> Express route -> controller -> model -> MongoDB
```

14. Project Status

14.1 Completed

- Static storefront pages and navigation
- Cart and wishlist persistence APIs
- Guest session identification flow
- Order creation and invoice generation backend
- Payment processing and verification backend schemas
- Secure Express backend with middleware
- PDF invoice service

14.2 In progress

- Frontend checkout UI is partially implemented in payment.js
- Backend order and payment flow exists, but full checkout page is not built

- Product and category APIs are not yet implemented

14.3 Pending

- User account, login, register, and profile pages
- Admin dashboard UI
- Real product backend CRUD endpoints
- Contact form submission endpoint
- Newsletter subscription endpoint
- Address book and review systems
- Admin product management and site settings

14.4 Future improvements

- Replace static product data with backend-driven catalog APIs
- Add robust authentication and session management
- Support multi-currency and multi-language storefronts
- Add analytics, telemetry, and monitoring
- Implement performance optimization and image CDN
- Add search indexing and backend search services
- Harden security for production deployment

15. Deployment Guide

15.1 Environment variables

Required backend env vars in `backend/.env`:

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/aljannat
JWT_SECRET=replace_with_secure_random_secret
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5000
SESSION_COOKIE_NAME=aljannat_session
SESSION_COOKIE_SECURE=false
RATE_LIMIT_WINDOW=15
RATE_LIMIT_MAX=100
```

15.2 Deploy on Docker

- Build: `docker build -t aljannat-backend backend`
- Run: `docker run -p 5000:5000 --env-file backend/.env aljannat-backend`
- Ensure MongoDB is accessible from container

15.3 Deploy on Render or Railway

- Push repository to GitHub
- Add Render/Railway service pointing to `backend` folder
- Configure environment variables
- Use Node 20 runtime
- Bind port to `process.env.PORT`

15.4 Deploy on VPS

- Install Node.js and MongoDB
- Clone repository
- Install backend dependencies in `backend`
- Start backend with `npm run start` or process manager such as PM2
- Serve frontend with static server or behind Nginx

15.5 Production checklist

- Set strong `JWT_SECRET`
- Use MongoDB auth and TLS
- Set `SESSION_COOKIE_SECURE=true`
- Use HTTPS for all traffic
- Enable rate limiting and security headers
- Add log retention and monitoring
- Use a CDN for assets and images

16. Maintenance Guide

16.1 Add new pages

- Create a new HTML file at repository root
- Add the shared header and footer structure
- Add page-specific `data-page` attribute
- Hook new page initialization in `js/app.js`
- Add route to backend static file serving if needed

16.2 Add new APIs

- Create a new route file in `backend/src/routes`
- Add controller logic in `backend/src/controllers`
- Define schema in `backend/src/models` if the collection is new
- Register route in `backend/src/index.js`

16.3 Add new products

- Add product objects to `js/products.js`
- Or implement backend product API and load products dynamically

16.4 Add payment gateways

- Add gateway-specific payment logic to `payment.controller.js`
- Extend payment schema with gateway transaction fields
- Update `payment.js` UI to collect gateway-specific data

16.5 Modify layouts

- Update CSS in `style.css` and `responsive.css`
- Keep shared classes consistent across pages
- Validate new layout changes in desktop and mobile breakpoints

16.6 Update the database

- Use Mongoose models to add or migrate fields
- Avoid direct schema changes in production without migration scripts
- Implement data backfill utilities if needed

16.7 Manage backups

- Backup MongoDB data regularly
- Store backups offsite or in cloud storage
- Keep static assets and uploads backed up along with the database

16.8 Scale the application

- Split frontend assets to a CDN
- Run backend behind a load balancer
- Scale MongoDB with replica sets or managed Atlas
- Add caching for product catalog and session data
- Monitor server and request latency

17. Visual Diagrams

17.1 System Architecture

```
[Browser] -> [Frontend Static Pages]
           -> [Backend Express API]
           -> [MongoDB Database]
```

17.2 API Flow

```
[Frontend] -> [storage.js] -> [API endpoint] -> [Express route] -> [Controller] -> [Model] -> [MongoDB]
```

17.3 Authentication Flow

```
[Login/Register Page] -> [POST /api/auth/login or register] -> [Auth Controller] -> [User Model] -> [JWT Token]
```

17.4 Cart Flow

```
[Add to Cart] -> [js/cart.js] -> [storage.js] -> PUT /api/cart/:productId -> cart.controller -> Cart model -> MongoDB
```

17.5 Checkout Flow

```
[Checkout Modal] -> [js/payment.js] -> POST /api/payments/process -> payment.controller -> Payment model -> /api/orders -> order.controller -> Order model -> Invoice service
```

17.6 Deployment Architecture

```
[User Browser] -> [CDN / Static Host] -> [Load Balancer] -> [Backend Node.js] -> [MongoDB]
```

18. Appendices

18.1 Notes on actual implementation

- Backend product endpoints are not present; the frontend currently relies on embedded static product data.
- Authentication frontend pages are not built.
- Invoice PDF generation is implemented but the frontend route for invoice download is not present.
- The backend currently serves static frontend pages from the repository root.

18.2 Recommended next phase

1. Implement product APIs and replace static js/products.js
2. Build login/register/profile UI and connect it to auth endpoints
3. Add a checkout page with backend order submission
4. Build admin UI for order and payment management
5. Add contact form submission and newsletter signup endpoints

End of document.